



Advanced Hibernate

1. Entity states
2. Transaction management
3. Hibernate caching

- An instance of a class that is managed by persistent context(OR Mapping tool) can be in different states.
- Transient
 - Exist in heap memory as normal Java objects
 - a transient entity has neither any representation in the datastore nor in the current Session
- Persistent or Managed
 - Persistent entities exist in the database mapped to a specific database row
 - All changes done to a managed entity can be reflected in the database.
 - Persistent entity can be created by saving transient or detached entity or by loading the entity from database.

- Detached
 - Detached entities have a representation in the database
 - But these are currently not managed by the Session.
 - Can be created by closing the session that it was associated with, or by evicting it from the session
 - A detached instance can be converted to persistent entity using `merge()`, `refresh()` etc
- Removed
 - When the persistent entities are removed using `delete` method, object is said to be in removed state.
 - It's corresponding row in the database is deleted.

- A transaction simply represents a unit of work.
- A transaction is a set of SQL operations that need to be either executed successfully or not at all.
- Transaction interface in hibernate helps in managing transactions.
- `session.beginTransaction()` helps in getting transaction associated with session.
- Methods of transaction which are commonly needed are:
 - `commit()`
 - `rollback()`
 - `isAlive()`
 - `setTimeout()`

- Caching is all about application performance optimization.
- Caching is between the application and the database to avoid the number of database hits.
- Two types of caching:
 - First level cache
 - Second-level cache
- First level cache
 - It is by default enabled by Hibernate itself.
 - The session object maintains the first-level cache
 - If we query for the same entity or object again with the same session object, it will be loaded from cache and no SQL query will be executed.

- Second level cache
 - The second-level cache is by default disabled.
 - The sessionfactory object maintains the first-level cache
 - The second-level cache is accessible by the entire application means data hold by SessionFactory can be accessible to all the sessions.
 - If the object needed by one session is already maintained in second level cache by some another session, it can be retrieved from sessionfactory without hitting the database.
 - Hibernate does not provide any default implementation for the second-level cache